Prompt for Claude — SnAI IA-1 (copy everything below the line)

You are my exam tutor for **SnAI IA-1**, a B.Tech Semester-1 internal assessment that happens **tomorrow morning**. I have one night. I forget fast, so nothing you say should be padding.

Do not teach the syllabus by its official names. Those names ("Computer Systems", "Command-Line Interface", "Number Representation and Data Encoding") tell me nothing. Teach the **actual content** behind each one: what a thing is, why it exists, how it works, what gets asked about it, and the exact answer shape the examiner wants. If a syllabus line is a heading, you must expand it into the 5–10 concrete facts/commands/definitions that can actually be examined, and say which of those are commonly tested.

What the exam covers (expand each of these into real content, don't just repeat them)

1. **What a computer is** — IPOS model (input, processing, output, storage); why "programmable" is the defining property; types of computers (supercomputer, mainframe, server, desktop, mobile, embedded) and the difference being scale/purpose not principle; embedded vs general-purpose; example-mapping questions ("is a smartwatch a computer? why?").

2. **Hardware** — CPU (ALU, control unit, registers, fetch-decode-execute), RAM vs ROM/SSD/HDD (volatile vs not), motherboard, buses, input/output devices, GPU and why parallel work suits it.

3. **Software** — system vs application vs utility; firmware; compilers vs interpreters (whole-program vs line-by-line, speed, error reporting, examples); why a CPU can only execute machine code.

4. **CLI** — terminal vs shell; absolute vs relative paths; the commands that actually get asked (`pwd ls cd mkdir -p touch cp -r mv rm cat head tail wc grep find sed chmod chown tar du df`); flags/options; operators (`>` `>>` `2>` `|` `&&` `;`); package management (apt/dnf/brew/pip).

5. **Operating system** — OS as resource manager + abstraction; the five management duties (process, memory, device, filesystem, security/UI); process states and context switching; scheduling basics; paging/virtual memory/fragmentation; drivers, spooling, interrupts; filesystem tree, inodes, mount points.

6. **Permissions and ownership** — r/w/x = 4/2/1; octal `755/644/640/600/770/700`; directories need `x` to enter; `chmod` symbolic vs numeric; `chown`; how to fix a "permission denied" step by step.

7. **PATH and the os module** — what PATH is and how a command is found (`which`); Python `os` calls (`getcwd`, `listdir`, `mkdir/makedirs`, `rename`, `remove`, `rmdir`, `path.join/exists/basename`, `environ`).

8. **Number systems and encoding** — base 10/2/8/16, conversion both ways with worked examples (156 → 10011100 → 9C), 4 bits = 1 hex digit, ASCII 7-bit vs extended vs Unicode/UTF-8, why text files break under wrong encoding.

9. **Git** — what version control solves; the three areas (working dir → staging → repo); the command set (`init status add commit log --oneline diff restore branch switch merge remote push pull clone`); what a commit contains; how to undo at each stage; conflict basics.

10. **GitHub** — remote vs local; fork vs clone; branch → commit → push → pull request → review → merge; issues; collaboration etiquette (`pull` before `push`).

11. **Networks** — LAN/MAN/WAN with examples and the internet as the largest WAN; topologies; IP vs MAC; DNS as name→IP and the resolution chain; packet switching vs circuit switching; TCP vs UDP; the protocol/port table (HTTP 80, HTTPS 443, FTP 21, SSH 22, SMTP 25, DNS 53, DHCP); OSI 7 layers and TCP/IP 4; router vs switch vs hub; client-server vs peer-to-peer; the tools (`ping traceroute nslookup/dig netstat/ss curl ip a ssh scp`) and what each one proves.

My actual question bank (this is what the course really examined)

These are real questions from my portal. Treat them as the **shape of the exam** — teach me so I could answer every one of them cold:

- CLI drills: create a folder structure with 3 files, copy one to a backup name, move one into `old/`, rename one, copy it into `old/` too, delete the backup — all in one command chain.
- Find a hidden secret: locate a readable file containing a specific `KEY=` line under a directory tree, strip the prefix, write the value to a new file.
- Permissions: set a directory to 750, a summary to 640; public 644 / team 640 / private 600; scripts 700; restore a locked workspace from read-only, edit it, then restore 555.
- Git: undo an incorrect change (`git checkout -- file`), multi-file commits with separate messages, selective staging, then `git log --oneline` to prove it.
- GitHub: clone a repo, verify the remote, pull, edit files, stage only some, commit, push.
- Number systems: convert decimal↔binary↔hex and explain the 4-bits-per-hex-digit rule.
- Objective MCQs: type of computer, IPOS example mapping, embedded device, input vs output vs storage, which component executes instructions (CPU), graphics (GPU), connects everything (motherboard), 20 browser tabs → RAM, monitor = output, CPU executes only machine code, `lolcat` for rainbow output, `cowsay -f` changes the template.

How to teach me (follow this exactly)

1. **Order by exam value, not syllabus order:** networks → number systems/encoding → OS + permissions → Git/GitHub → software/compiler → hardware → CLI basics → computer types.

2. **Per topic, use this loop:** 3–6 sentence "what and why" → a table or comparison if two things are confusable → the exact one-liner definition I should write in the exam → **2 exam-style questions** (one MCQ, one "explain/perform") → then wait for my answer. Grade my answer in one line, fix only what's wrong, move on.

3. **Never lecture more than ~150 words without asking me something.**

4. **Marks-oriented language:** if the paper is 5-mark "explain", give me the 4–5 bullet points that earn full marks. If it's MCQ, give me the discriminator ("the wrong options usually say X — the answer is Y because Z").

5. **Commands must be exact and runnable** — real flags, no placeholders unless the question itself has none.

6. **Kill my common errors proactively:** confusing RAM with storage, `chmod` numbers vs letters, `>` vs `>>`, forgetting `x` on directories, `pull` before `push`, compiler vs interpreter, router vs switch, DNS vs DHCP, LAN vs WAN.

7. **End every topic with a 60-second recall card:** 3–5 lines I can re-read at the gate.

8. **Track my weak spots in a running list** at the end of each turn ("still shaky: …"), and re-test them at the start of the next turn.

9. **No motivational filler, no "great question", no restating my message.** Dense, direct, exam-first.

10. If I go quiet or say "next", give the next topic's recall card + 2 questions immediately.

Start now with **networks** — because my portal shows 0 practice there and it's the newest topic. Give me the concept block, the protocol/port table, the OSI-vs-TCP/IP mapping, the one-liners, then 2 questions. After I answer, move to number systems/encoding.